ZAMP · AI SOLUTIONS ANALYST CASE STUDY · PS-1 FINANCE / AP

InvoiceFlow Agent

Build direction and specification: invoice PDF in, explained decision out

Version 1.0 · 11 Nov 2026 · Deadline: Thu 12 Nov 2026, 23:59 IST
Stack: Python · Streamlit · Groq LLM · SQLite · Streamlit Community Cloud

> **How to use this document.** This is the contract for the build. Anyone helping (human or AI) follows it exactly. Schemas, file ownership, reason codes, thresholds and seed data are fixed. To change something, update this document first, then the code. Section 17 has a ready-to-paste prompt for AI helpers.

Contents

1. Goal and success criteria
2. Golden rules (never break)
3. Assumptions
4. Tech stack
5. Architecture
6. Pipeline stages
7. Data model (SQLite)
8. Pydantic schemas
9. Seed data (fixed values)
10. Rules, thresholds, reason codes
11. The four agents
12. Edge cases and expected outputs
13. Files and function ownership
14. UI specification
15. Build plan and checkpoints
16. Deploy, demo script, risks
17. Prompt for AI helpers

1. Goal and success criteria

A live, hosted web app. An AP user uploads a vendor invoice PDF (digital or scanned). The system extracts the data, checks it against vendor and purchase-order records, applies explicit AP rules, and returns one decision with reasons, an explanation and a next action. Every stage shows live as it runs, and every run is stored for the dashboard.

Requirement from Zamp guide	How we meet it
Process actually runs on real inputs	Upload a real PDF → real Groq calls → real rule evaluation → stored result
Live run view (each stage as it executes)	Orchestrator yields events. Streamlit renders 10 stage cards that update live.
Dashboard (history, status, outputs)	Dashboard page: KPI tiles, run table, filters, drill-down into any run
2–4 non-trivial edge cases	Split-PO overrun, scanned duplicate, missing PO (agent inference), bank-detail change
Decision explained	Reason codes + evidence table + plain-English narrative
Live and runnable link	Streamlit Community Cloud public URL
5-minute demo video	Script in section 16

2. Golden rules (never break)

1. **The LLM extracts and reasons. Only `rules.py` decides.** No LLM output may set the final decision.
2. **Every LLM output gets validated by Pydantic.** On failure: retry once with the error, then fall back. Never crash the run.
3. **Every stage emits a `StepEvent`** (status, summary, detail) and gets written to `run_steps`.
4. **Missing data never gets guessed.** Null stays null and goes to HUMAN_REVIEW.
5. **All money uses `Decimal`**, rounded to 2 places, and displays in Indian format (₹4,13,000.00).
6. **Thresholds live only in `config.py`.** No magic numbers in other files.
7. **A "Reset demo data" button** must always restore the seed state. Demos must work in any order.
8. **Happy path first.** No edge-case work until the happy path runs end to end on the live link.

3. Assumptions (copy into ASSUMPTIONS.md)

- Buyer company: **Zenith Retail Pvt Ltd**, Bengaluru, GSTIN `29AABCZ1234F1Z5` (state code 29, Karnataka). All data is fictional.
- Currency is INR only. GST rate is 18% for all goods and services (a simplification).
- Intra-state vendor (GSTIN state = 29) charges CGST 9% + SGST 9%. Inter-state charges IGST 18%.
- Two-way match (invoice vs PO). Goods receipt (three-way match) is out of scope and noted as a next step.
- One invoice per PDF, at most 3 pages (Groq vision limit: 3 images per request).
- Input arrives via upload. Email ingestion is out of scope and noted as a next step.
- "Billed to date" on a PO counts only invoices with status APPROVE, APPROVE_WITH_NOTE or PAID.
- GSTIN gets a format check only (regex). No live GST portal lookup.

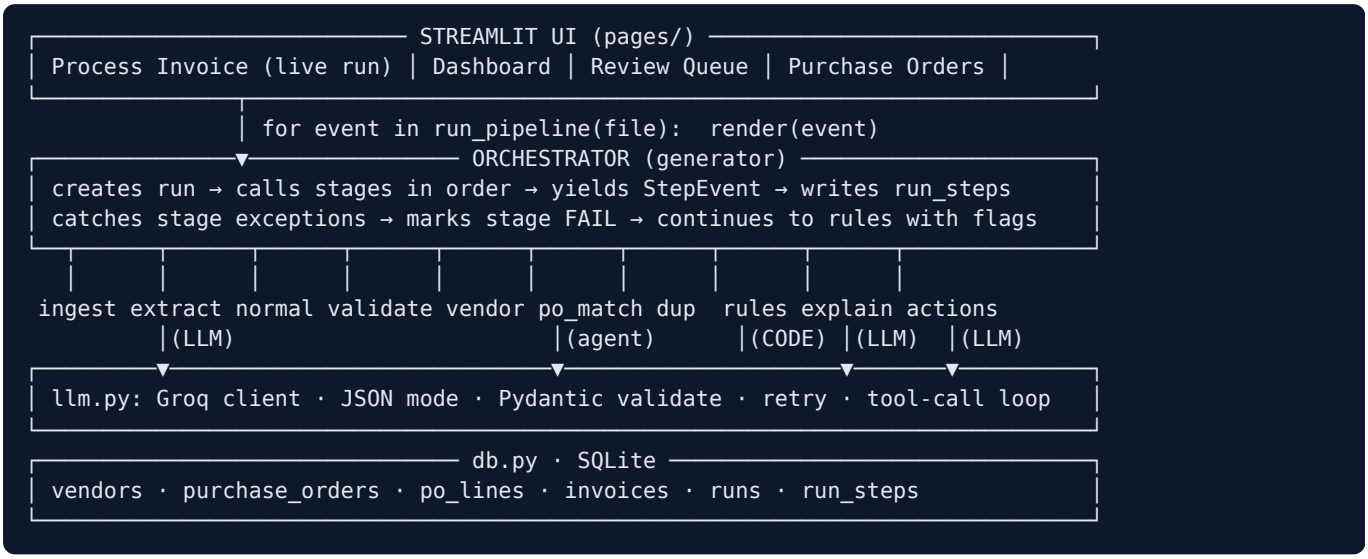
4. Tech stack

Layer	Choice	Why
Language	Python 3.11+	The only language the builder codes in
UI + hosting	Streamlit + Community Cloud	Pure Python. `st.status` gives live stage cards. Free public URL.
LLM	Groq API (`groq` SDK)	Key already available. Very fast, so the live demo feels instant.
Text model	e.g. `llama-3.3-70b-versatile`	Digital PDFs, PO-inference tool calls, narratives
Vision model	Current Groq vision model (check docs)	Scanned PDFs. Groq's vision lineup changes often, so confirm the name at console.groq.com/docs/vision and set it in config.
PDF text	`pdfplumber`	Reads text from digital PDFs. Empty text means scanned.
PDF → image	`pypdfium2` + `Pillow`	Renders pages to PNG for vision. No system dependencies, so it works on Streamlit Cloud.
Fuzzy matching	`rapidfuzz`	Vendor names, invoice numbers, line descriptions
Validation	`pydantic` v2	Strict schemas for all LLM output
Storage	SQLite (stdlib `sqlite3`)	Zero setup. Gets re-seeded on boot and on reset.
Tables/charts	`pandas` + Streamlit native charts	Dashboard

Test invoices	`reportlab`	Script generates all sample PDFs, including the scanned one

requirements.txt: streamlit, groq, pydantic, pdfplumber, pypdfium2, pillow, rapidfuzz, pandas, reportlab, python-dotenv. Secrets: `GROQ_API_KEY` in `.streamlit/secrets.toml` locally and in the Cloud secrets UI. Never commit it.

5. Architecture



Key design idea for the pitch: "AI reads and reasons. Rules decide. Humans override."

Ingest → Extract (AI) → Normalize → Validate → Vendor → PO Match (AI agent) → Duplicate → Rules → Decision → Explain (AI) → Action (AI draft)

Purple = LLM involved. Orange = deterministic decision point. Navy = plain code.

6. Pipeline stages

Each stage is a function `(ctx: RunContext) -> StepResult`. It reads from and writes to the shared `RunContext`. It never raises past the orchestrator.

#	Stage / file	Input → Output	Logic	On failure
1	Ingest `ingest.py`	bytes → file_hash, page_count, text, is_scanned, page_images	SHA-256. pdfplumber text. If text < 50 chars → scanned → render pages at 150 dpi.	FAIL run: "unreadable file"
2	Extract `extract.py`	text or images → `ExtractedInvoice`	Text model if digital, vision model if scanned. JSON mode. Pydantic validate. Retry once.	FAIL → HUMAN_REVIEW (EXTRACTION_FAILED)
3	Normalize `normalize.py`	raw fields → clean fields	Dates to ISO. Amounts to Decimal (strip ₹, commas). Invoice no. normalized (uppercase alphanumerics only). GSTIN uppercase.	Unparseable value becomes null + WARN
4	Validate `validate.py`	invoice → flags	Required fields present. GSTIN regex. Σ lines = taxable (±₹1). taxable + tax = total (±₹1). Tax type matches state. Confidence ≥ 0.70 on critical fields.	—

5	Vendor `vendor.py`	invoice → vendor record, flags	Match by GSTIN exact, else name fuzzy ≥ 88. Check status APPROVED. Compare bank account + IFSC with master.	No match → VENDOR_UNKNOWN
6	PO match `po_match.py`	invoice + vendor → po_id, match_method, confidence, line matches	Explicit PO ref found in DB → use it. Else the PO inference agent (section 11) runs, then a deterministic score verifies its answer.	No PO → PO_NOT_FOUND
7	Duplicate `duplicate.py`	invoice → dup flags	Same file hash → exact. Same vendor + same normalized invoice no. → exact. Same vendor + total ±₹1 + date within 7 days → possible.	—
8	Rules `rules.py`	all flags + numbers → `Decision`	Price/qty variance per line. Cumulative billed vs PO. Map flags to reason codes. Apply precedence.	Must never fail (pure code, unit tested)
9	Explain `explain.py`	Decision + evidence → narrative	LLM writes 3–5 sentences using only the evidence numbers.	Fallback: template text built from the reason codes
10	Action `actions.py`	Decision → action record	APPROVE → mock ledger entry. HOLD → vendor email draft. HUMAN_REVIEW → review queue (+ internal alert if bank changed). REJECT → vendor notice draft.	Fallback: template draft

7. Data model (SQLite)

```
vendors(vendor_id TEXT PK, name TEXT, gstin TEXT, state_code TEXT, bank_account TEXT,
        bank_ifsc TEXT, status TEXT  -- APPROVED | BLOCKED
        , contact_email TEXT, contact_phone TEXT)

purchase_orders(po_id TEXT PK, vendor_id TEXT FK, po_date TEXT, status TEXT  -- OPEN | CLOSED
        , total_taxable REAL, description TEXT)

po_lines(po_id TEXT FK, line_no INT, description TEXT, quantity REAL, unit_price REAL, amount REAL)

invoices(invoice_id INTEGER PK, run_id TEXT NULL, vendor_id TEXT, invoice_number TEXT,
        invoice_number_norm TEXT, invoice_date TEXT, po_id TEXT, taxable_value REAL,
        total_amount REAL, file_hash TEXT, status TEXT, source TEXT  -- SEED | RUN
        , created_at TEXT)

runs(run_id TEXT PK, filename TEXT, file_hash TEXT, started_at TEXT, duration_ms INT,
        decision TEXT, reason_codes TEXT/*JSON*/, headline TEXT, extracted TEXT/*JSON*/,
        evidence TEXT/*JSON*/, explanation TEXT, action TEXT/*JSON*/,
        review_status TEXT  -- NONE | PENDING | OVERRIDDEN_APPROVE | OVERRIDDEN_REJECT | CONFIRMED
        , review_note TEXT, reviewed_at TEXT)

run_steps(run_id TEXT, seq INT, stage TEXT, status TEXT  -- OK | WARN | FAIL | SKIP
        , duration_ms INT, summary TEXT, detail TEXT/*JSON*/)
```

`db.py` owns: `init_db()`, `reset_and_seed()`, `get_vendor_by_gstin()`, `search_vendors()`, `get_po()`, `get_po_lines()`, `list_open_pos(vendor_id)`, `get_billed_to_date(po_id)`, `find_invoices(...)`, `insert_run()`, `insert_step()`, `insert_invoice()`, `update_review()`, `list_runs()`, `get_run()`.

8. Pydantic schemas (`models.py`)

```
class LineItem(BaseModel):
    description: str | None; hsn_sac: str | None
    quantity: Decimal | None; unit_price: Decimal | None; amount: Decimal | None

class ExtractedInvoice(BaseModel):
    invoice_number: str | None;  invoice_date: str | None;  due_date: str | None
    vendor_name: str | None;     vendor_gstin: str | None;  buyer_gstin: str | None
    po_reference: str | None
    line_items: list[LineItem] = []
    taxable_value: Decimal | None
    cgst: Decimal | None; sgst: Decimal | None; igst: Decimal | None
    total_amount: Decimal | None; currency: str | None = "INR"
    bank_account_number: str | None; bank_ifsc: str | None
    field_confidence: dict[str, float] = {}   # 0..1, keys = field names above
    extraction_notes: str | None              # anything odd the model noticed

class StepEvent(BaseModel):
    run_id: str; seq: int; stage: str
    status: Literal["RUNNING","OK","WARN","FAIL","SKIP"]
    summary: str; detail: dict = {}; duration_ms: int = 0

class Decision(BaseModel):
    status: Literal["APPROVE","APPROVE_WITH_NOTE","HOLD","HUMAN_REVIEW","REJECT"]
    reason_codes: list[str]; headline: str       # one-line summary
    evidence: dict                               # numbers used by rules
    approvable_amount: Decimal | None = None     # e.g. split-PO partial

class RunContext:   # plain dataclass, mutable, passed through all stages
    run_id, filename, file_bytes, file_hash, text, is_scanned, page_images,
    extracted, vendor, po, po_lines, line_matches, match_method, match_confidence,
    flags: list[Flag], decision, explanation, action
```

Confidence rule: self-reported LLM confidence is unreliable, so the code computes its own. `effective_conf(field) = min(llm_conf, 1.0 if format_valid else 0.4)`. Critical fields: invoice_number, invoice_date, vendor_gstin, total_amount, taxable_value.

9. Seed data (fixed values: do not change)

These values are what make the edge cases trigger. Store them in `data/seed/*.json`, loaded by `reset_and_seed()`.

Vendors

ID	Name	GSTIN	State	Bank (IFSC / account)	Status
V001	Acme Office Supplies Pvt Ltd	29AAACA1111A1Z1	29 KA (intra)	HDFC0001234 / 50100012345678	APPROVED
V002	Bharat Cloud Services LLP	27AAJFB2222B1Z2	27 MH (inter)	ICIC0000456 / 000405012345	APPROVED
V003	Sharma Logistics	29ABCPS3333C1Z3	29 KA (intra)	SBIN0001111 / 30012345678	APPROVED
V004	Nimbus Print Works	33AADCN4444D1Z4	33 TN (inter)	UTIB0002222 / 912010012345	BLOCKED

Contacts: V001 accounts@acme-office.in, +91 80 4000 1111 · V002 billing@bharatcloud.in, +91 22 6000 2222 · V003 ops@sharmalogistics.in, +91 80 4111 3333 · V004 hello@nimbusprint.in.

Purchase orders

PO	Vendor	Lines	Taxable total	Status
PO-2026-0101	V001	1) Ergonomic office chair, 50 × ₹4,000 = ₹2,00,000 2) Office desk 120cm, 20 × ₹7,500 = ₹1,50,000	₹3,50,000	OPEN
PO-2026-0102	V002	1) Managed cloud hosting Q4 (Oct–Dec), 3 months × ₹2,00,000 = ₹6,00,000	₹6,00,000	OPEN
PO-2026-0103	V003	1) Freight Bengaluru–Chennai (FTL), 12 trips × ₹15,000 = ₹1,80,000	₹1,80,000	OPEN
PO-2026-0104	V001	1) A4 copier paper 75gsm (ream), 100 × ₹300 = ₹30,000	₹30,000	OPEN
PO-2026-0105	V003	1) Warehouse loading/unloading labour, 40 shifts × ₹3,000 = ₹1,20,000	₹1,20,000	OPEN

Invoice history (source = SEED)

Vendor	Invoice no.	Date	PO	Taxable	Total (IGST 18%)	Status
V002	BCS/2026/1001	2026-10-05	PO-2026-0102	₹2,00,000	₹2,36,000	PAID
V002	BCS/2026/1102	2026-11-04	PO-2026-0102	₹2,00,000	₹2,36,000	APPROVE

Sample invoices to generate (`scripts/generate_invoices.py`)

File	Content	Expected
01_happy_acme.pdf	Acme, ACME/INV/0057, 2026-11-08, PO-2026-0101, chairs 50×4,000 + desks 20×7,500, taxable 3,50,000, CGST 31,500, SGST 31,500, total 4,13,000, bank HDFC0001234 / 50100012345678	APPROVE
02_split_po_overrun.pdf	Bharat, BCS/2026/1203, 2026-12-03, PO-2026-0102, hosting Dec 1×2,00,000 + "Additional bandwidth 2TB" 1×40,000, taxable 2,40,000, IGST 43,200, total 2,83,200	HOLD
03_scanned_duplicate.pdf	**Scanned image** (slight rotation and noise). Bharat, printed number "BCS-2026-1102", 2026-11-04, PO-2026-0102, taxable 2,00,000, IGST 36,000, total 2,36,000. Different layout from the original.	REJECT
04_missing_po_sharma.pdf	Sharma, SL/26-27/0318, 2026-11-10, **no PO reference**, "Freight charges – Bengaluru to Chennai, full truck load, Oct–Nov 2026" 12×15,000, taxable 1,80,000, CGST 16,200, SGST 16,200, total 2,12,400	APPROVE_WITH_NOTE
05_bank_change_acme.pdf	Acme, ACME/INV/0061, 2026-11-09, PO-2026-0104, paper 100×300, taxable 30,000, CGST 2,700, SGST 2,700, total 35,400. Footer: "Please note our **new bank details**: Kotak Mahindra Bank, IFSC KKBK0000789, A/c 9988776655"	HUMAN_REVIEW
06_blocked_vendor.pdf (optional)	Nimbus Print Works, NPW/112, no PO, taxable 25,000, IGST 4,500, total 29,500	REJECT

Scanned PDF recipe: build the PDF with reportlab → render with pypdfium2 at 110 dpi → Pillow: grayscale, rotate 1.2°, add light Gaussian noise, slight blur → save as an image-only PDF. Verify that pdfplumber returns < 50 chars for it.

10. Rules, thresholds, reason codes (`rules.py`, `config.py`)

Thresholds (config.py)

```
BUYER_GSTIN = "29AABCZ1234F1Z5";  BUYER_STATE = "29";  GST_RATE = Decimal("0.18")
ROUNDING_TOLERANCE_INR      = Decimal("1.00")
PRICE_VARIANCE_OK_PCT       = Decimal("1.0")   # ≤1%  → pass
PRICE_VARIANCE_NOTE_PCT     = Decimal("3.0")   # 1–3% → APPROVE_WITH_NOTE ; >3% → HOLD
PO_OVERRUN_TOLERANCE_PCT    = Decimal("1.0")   # cumulative billed may exceed PO by ≤1%
MIN_FIELD_CONFIDENCE        = 0.70
VENDOR_NAME_FUZZY_MIN       = 88
PO_INFER_ACCEPT_CONF        = 0.80             # ≥0.80 → accept with note ; else HUMAN_REVIEW
DUP_DATE_WINDOW_DAYS        = 7
GSTIN_REGEX = r"^[0-9]{2}[A-Z]{5}[0-9]{4}[A-Z][1-9A-Z]Z[0-9A-Z]$"
```

Reason codes → decision

Code	Trigger	Decision
VENDOR_BLOCKED	Vendor status ≠ APPROVED	REJECT
DUPLICATE_EXACT	Same hash, or same vendor + normalized invoice no.	REJECT
BANK_DETAILS_CHANGED	Invoice bank account or IFSC ≠ vendor master	HUMAN_REVIEW
DUPLICATE_POSSIBLE	Same vendor + total ±₹1 + date within 7 days, different number	HUMAN_REVIEW
MISSING_CRITICAL_FIELD	A critical field is null	HUMAN_REVIEW
LOW_CONFIDENCE	Critical field effective confidence < 0.70	HUMAN_REVIEW
VENDOR_UNKNOWN	No GSTIN or name match	HUMAN_REVIEW
PO_NOT_FOUND	No explicit PO and inference failed	HUMAN_REVIEW
PO_INFERRED_LOW_CONF	Inferred PO confidence < 0.80	HUMAN_REVIEW
EXTRACTION_FAILED	LLM output invalid after retry	HUMAN_REVIEW
PO_OVERRUN	billed_to_date + taxable > PO total × 1.01	HOLD
PRICE_VARIANCE_HIGH	Any line unit-price variance > 3%	HOLD
UNMATCHED_LINE	Invoice line with no PO line match (fuzzy < 70)	HOLD
ARITHMETIC_MISMATCH	Σ lines ≠ taxable, or taxable + tax ≠ total (beyond ₹1)	HOLD
TAX_TYPE_MISMATCH	Intra-state but IGST charged, or vice versa, or rate ≠ 18%	HOLD
PO_CLOSED	Matched PO status = CLOSED	HOLD
PRICE_VARIANCE_MINOR	Line variance between 1% and 3%	APPROVE_WITH_NOTE
PO_INFERRED	PO inferred by agent with confidence ≥ 0.80	APPROVE_WITH_NOTE

> **Precedence:** REJECT > HUMAN_REVIEW > HOLD > APPROVE_WITH_NOTE > APPROVE. The final status is the most severe triggered code. All triggered codes get stored and shown. If no codes trigger → APPROVE.
> **Partial approval hint:** on PO_OVERRUN, set `approvable_amount = PO total − billed_to_date`, shown in the decision card.

`rules.py` functions: `line_variances(invoice_lines, po_lines) → list`, `check_po_capacity(po, billed, taxable) → Flag|None`, `collect_flags(ctx) → list[Flag]`, `decide(flags, evidence) → Decision`. Pure functions with no I/O, so they can be unit tested.

11. The four agents (where AI earns its place)

A. Extraction agent (`extract.py`)

- Input: page text (digital) or up to 3 page images (scanned). Output: `ExtractedInvoice` JSON (JSON mode, temperature 0).
- Prompt contract: "Extract only what is printed. If a field is absent, return null. Never compute or infer values. Report per-field confidence 0–1. Put anything unusual (e.g. a bank-change notice, handwritten notes) in extraction_notes."
- Include the JSON schema and one short example in the prompt. Validate. On failure, resend with the validation error once.

B. PO inference agent (`po_match.py`): the visibly agentic part

- Runs only when there is no usable PO reference. Tool-calling loop, max 4 turns, temperature 0.
- Tools exposed to the model: `list_open_pos(vendor_id)` · `get_po_lines(po_id)` · `get_billed_to_date(po_id)`
- Must return JSON: `{"po_id": str|null, "confidence": 0..1, "rationale": str}`.
- **Deterministic verification** (code, not LLM): score = 0.6 × best line-description fuzzy match + 0.4 × amount fit (invoice taxable ≤ PO remaining). Final confidence = min(agent confidence, score). The UI shows the tool calls the agent made as a trace.
- Fallback if tool-calling errors: pure deterministic scoring across the vendor's open POs.

C. Explainer (`explain.py`)

- Input: Decision + evidence. Output: 3–5 sentences for an AP manager. "Use only numbers present in the evidence. Do not change the decision. Say what to do next." Fallback: a template per reason code.

D. Action drafter (`actions.py`)

- HOLD → email draft to the vendor (what's wrong + exactly what's needed, e.g. "a revised invoice for ₹2,00,000 or a PO amendment for ₹40,000").
- HUMAN_REVIEW with bank change → **internal** alert to the AP lead: "Do not pay. Verify by calling the number on file (+91 80 4000 1111), not contacts on the invoice." Never email the vendor about a bank change.
- REJECT → short notice to the vendor. APPROVE → mock ledger entry (JSON: GL code, amount, due date).

12. Edge cases: design and expected outputs

Case	Why it's realistic	What the system must do
1. Split-PO overrun 02_split_po_overrun.pdf	Vendors bill one PO across months. Each invoice looks fine on its own, but the overrun only shows when you track cumulative billing. Plus scope creep (bandwidth not on the PO).	Billed to date ₹4,00,000 + ₹2,40,000 = ₹6,40,000 vs PO ₹6,00,000 (+6.7%). Codes: PO_OVERRUN, UNMATCHED_LINE → HOLD. Approvable ₹2,00,000. Vendor draft asks for a revised invoice or a PO amendment for ₹40,000.
2. Scanned duplicate 03_scanned_duplicate.pdf	Vendors re-send unpaid invoices as scans with reformatted numbers. Exact-match checks miss them. Double payment is a top AP loss.	Scanned path → vision model. "BCS-2026-1102" normalizes to the same value as the seeded "BCS/2026/1102" → DUPLICATE_EXACT → REJECT. Evidence shows the original invoice and its status.
3. Missing PO reference 04_missing_po_sharma.pdf	Service vendors often omit the PO. A human would search the vendor's open POs. The vendor has two open POs, so the choice is non-trivial.	The agent calls tools, picks PO-2026-0103 over 0105 (description + amount fit), confidence ≥ 0.8 → PO_INFERRED → APPROVE_WITH_NOTE. The tool-call trace is visible.

4. Bank details changed 05_bank_change_acme.pdf	Business email compromise: a known, trusted vendor "updates" bank details. Everything else matches perfectly. This is the most common AP fraud pattern.	Vendor, PO and amounts all pass, but bank ≠ master → BANK_DETAILS_CHANGED → HUMAN_REVIEW. Payment blocked. Internal verify-by-phone alert. Shows the rules override a "perfect" match.

13. Files and function ownership

```
invoiceflow/
├── app.py                          # Page 1 "Process Invoice": upload, sample picker, live run view
├── pages/
│   ├── 1_Dashboard.py              # KPIs, run history table, filters, run drill-down
│   ├── 2_Review_Queue.py           # HUMAN_REVIEW + HOLD items; approve/reject override + note
│   └── 3_Purchase_Orders.py        # PO list, billed vs remaining progress bars, linked invoices
├── core/
│   ├── config.py                   # thresholds, model names, buyer info, secrets loading
│   ├── models.py                   # Pydantic schemas + RunContext + Flag dataclass
│   ├── db.py                       # SQLite schema, seed/reset, all queries (no SQL elsewhere)
│   ├── llm.py                      # chat_json(), chat_vision_json(), run_tool_loop();
retry/validate
│   ├── prompts.py                  # EXTRACT_TEXT, EXTRACT_VISION, PO_AGENT, EXPLAIN, DRAFT_*
prompts
│   ├── money.py                    # to_decimal(), format_inr(), pct_diff()
│   └── pipeline/
│       ├── orchestrator.py         # run_pipeline(filename, bytes) -> Iterator[StepEvent]
│       ├── ingest.py               # ingest(ctx): hash, text, is_scanned, render_pages()
│       ├── extract.py              # extract(ctx)
│       ├── normalize.py            # normalize(ctx): norm_date(), norm_invoice_no(), norm_gstin()
│       ├── validate.py             # validate(ctx): required, gstin, arithmetic, tax_type,
confidence
│       ├── vendor.py               # vendor_check(ctx): match_vendor(), bank_changed()
│       ├── po_match.py             # po_match(ctx): match_explicit(), infer_po_agent(), score_po()
│       ├── duplicate.py            # duplicate_check(ctx)
│       ├── rules.py                # collect_flags(), decide()  ← ONLY place a decision is made
│       ├── explain.py              # explain(ctx)
│       └── actions.py              # act(ctx): ledger_entry(), draft_vendor_email(),
internal_alert()
├── ui/
│   └── components.py               # stage_card(), decision_card(), status_pill(), evidence_table()
├── data/seed/                      # vendors.json, purchase_orders.json, invoice_history.json
├── samples/                        # 01..06 PDFs (generated)
├── scripts/
│   ├── generate_invoices.py        # builds all sample PDFs incl. scanned
│   └── smoke_test.py               # runs all samples via orchestrator, asserts expected decisions
├── tests/test_rules.py             # unit tests for rules.decide() and variances
├── .streamlit/config.toml          # theme (primary #2f5bea), wide layout
├── requirements.txt · README.md · ASSUMPTIONS.md · .gitignore (secrets.toml, *.db)
```

Orchestrator contract

```
STAGES = [("Ingest", ingest), ("Extract", extract), ("Normalize", normalize),
          ("Validate", validate), ("Vendor check", vendor_check), ("PO match", po_match),
          ("Duplicate check", duplicate_check), ("Rules & decision", decide_stage),
          ("Explain", explain), ("Action", act)]

def run_pipeline(filename, file_bytes) -> Iterator[StepEvent]:
    ctx = RunContext.new(filename, file_bytes); db.insert_run(ctx)
    for seq, (name, fn) in enumerate(STAGES, 1):
        yield StepEvent(stage=name, status="RUNNING", ...)
        t0 = now()
        try:    res = fn(ctx)                    # returns StepResult(status, summary, detail)
        except Exception as e:  res = StepResult("FAIL", f"{name} failed: {e}", {})
                                ctx.flags.append(Flag("STAGE_FAILED", stage=name))
        db.insert_step(...); yield StepEvent(stage=name, status=res.status, ...)
        if name == "Ingest" and res.status == "FAIL": break   # nothing to process
    db.finalize_run(ctx)   # decision, explanation, action; insert into invoices if not REJECT
```

14. UI specification

Page 1: Process Invoice (`app.py`)

- Header: product name + one-line tagline. Sidebar: "Reset demo data" button, model names, link to the Assumptions section.
- Input: file uploader **and** a "Try a sample" selectbox (the 6 samples). Run button.
- Left (60%): 10 stage cards using `st.status`, each updating live: icon, name, duration, one-line summary, expandable detail (extracted JSON, matches, tool-call trace, flags).
- Right (40%): PDF preview (first page image) → after the run, the **Decision card**: coloured status pill, headline, reason-code chips, key numbers (invoice total, PO total, billed to date, variance), explanation, action draft (st.code, with copy).

Page 2: Dashboard

- KPI tiles: total runs · auto-approved % · ₹ processed · ₹ on hold · pending reviews · avg processing time.
- Bar chart: decisions by status. Table: time, file, vendor, invoice no., total, decision pill, codes, duration. Filter by status and vendor.
- Select a row → full run detail: stored stage timeline + decision card (same components).

Page 3: Review Queue

- Lists HUMAN_REVIEW and HOLD runs with review_status PENDING. Each shows the reason, the evidence and the draft. Buttons: Approve override / Reject / Confirm hold. Requires a note. Gets logged with a timestamp. Shows human in the loop.

Page 4: Purchase Orders

- Per PO: vendor, total, billed to date, remaining, progress bar (red if over), linked invoices. Makes the split-PO story visible.

15. Build plan and checkpoints

Time (IST)	Work	Checkpoint (must be true)
Wed 23:15–00:45	Email coordinator ("ASA Case Study — Name — PS-1"). Create GitHub repo + folder skeleton. Test the Groq key with one call (text + vision). Deploy a "hello" Streamlit app to Community Cloud with the secret set.	Public URL loads. Groq call works from the cloud.

Wed 00:45–07:30	**Sleep.**	Non-negotiable
Thu 08:00–10:00	config, models, money, db (schema + seed + reset). generate_invoices.py → all 6 PDFs.	DB seeds. 6 PDFs open correctly. The scanned one has no text.
Thu 10:00–13:30	llm.py, prompts, ingest → extract → normalize → validate → vendor → po_match (explicit only) → duplicate → rules → explain → action. Orchestrator. Run from the CLI.	**Happy path gives APPROVE from the CLI.** test_rules.py passes.
Thu 13:30–14:00	Break	
Thu 14:00–16:30	app.py live run view + decision card. Dashboard page. Push → test on the live URL.	Happy path runs live on the public URL.
Thu 16:30–19:00	Edge cases in order: 4 (bank) → 2 (duplicate/scan) → 1 (split PO) → 3 (agent). Review Queue + PO pages. smoke_test.py.	All 4 edge cases give their expected decisions on the live URL.
Thu 19:00–20:00	UI polish, error messages, README, ASSUMPTIONS.	
Thu 20:00–21:00	Dinner / buffer for bugs	
Thu 21:00–22:30	Reset data. Rehearse twice. Record the 5-min video (Loom).	Video ≤ 5:00, uploaded, link works logged out.
Thu 22:30–23:30	Submit: live URL + video link (+ GitHub). Buffer.	Submitted before 23:30.

> **Cut list if behind (cut in this order):** 06 blocked vendor sample → PO page → review-queue overrides (keep the list only) → PO agent tool loop (keep deterministic scoring and call it "matcher") → LLM explainer (use templates). **Never cut:** live run view, dashboard, happy path + 3 edge cases, reset button.

16. Deploy, demo script, risks

Deploy (Streamlit Community Cloud)

- Public GitHub repo → share.streamlit.io → New app → `app.py` → Advanced → Secrets: `GROQ_API_KEY="..."`.
- The SQLite file is ephemeral on Cloud: `init_db()` seeds automatically if empty. Run history resets on reboot, which is acceptable. Mention it as a known limitation (next step: Postgres).
- Open the app 10 minutes before any demo, since it sleeps after inactivity.

5-minute demo video script

Time	Show / say
0:00–0:30	Problem in one line. Design principle: "AI reads and reasons, rules decide, humans override."
0:30–1:30	Happy path: upload 01 → stages light up live → APPROVE. Open the Extract and PO match details.
1:30–2:20	Bank change (05): everything matches, yet HUMAN_REVIEW. Show the internal alert. "This is the fraud case."
2:20–3:10	Split PO (02): HOLD, billed vs PO numbers, ₹2,00,000 approvable, vendor email draft. Flash the PO page.
3:10–3:50	Missing PO (04): agent tool-call trace, inferred PO, deterministic verification score.
3:50–4:20	Scanned duplicate (03): vision path, REJECT with a link to the original.
4:20–5:00	Dashboard + review queue override. Next steps: email ingestion, 3-way match with GRN, ERP posting, learning vendor templates.

Risks and mitigations

Risk	Mitigation

Groq rate limit / outage mid-demo	Retry with backoff. Cache extraction by file hash (same file → reuse JSON). Mention the cache in the demo as a cost saver.
Vision model renamed/deprecated	Model name only in config. Verify at the start of the build.
LLM JSON invalid	JSON mode + Pydantic + 1 retry + HUMAN_REVIEW fallback
Agent picks the wrong PO	Deterministic score caps the confidence. Low confidence → human review.
Deploy breaks late	Hello-world deployed tonight. Push and test every checkpoint.
Demo order dependency	Seeded history + Reset button. Any sample works in any order.

17. Prompt for any AI helper (paste with this PDF)

```
You are helping build "InvoiceFlow Agent", a Streamlit + Groq + SQLite app for
invoice processing (PDF → explained AP decision). The attached Direction PDF is
the binding specification. Rules for you:

1. Follow the spec exactly: file names, function names, schemas, reason codes,
   thresholds and seed values. Do not invent new files, fields or codes.
2. The LLM never decides. Only core/pipeline/rules.py sets Decision.status.
3. Python only. Use Decimal for money. Thresholds only in core/config.py.
   SQL only in core/db.py. Prompts only in core/prompts.py.
4. Write ONE file at a time, complete and runnable, no placeholders or "..." .
   State which spec section the file implements.
5. If the spec is ambiguous, say so and propose a choice. Do not silently
   deviate. If you think the spec is wrong, argue it, then wait.
6. Every stage function signature: def stage(ctx: RunContext) -> StepResult.
   Stages must not raise; the orchestrator catches errors.
7. After each file, give a 1-line command to test it.

Current task: <describe the one file or fix you need>
Current state: <paste relevant existing code or error>
```

> **Definition of done:** the public URL processes all 5 core samples with their expected decisions, the dashboard shows the runs, the reset works, and a ≤ 5-minute video is linked and submitted before midnight.